AI Engineer — 1-Month Roadmap Checklist

~3-4 hrs/day (≈100 hrs) · application layer + fine-tuning · goal: job-ready portfolio + interviews

★ = *do-not-skip, highest-leverage phases.* *Full detail in ai_engineer_roadmap.md; resources in ai_engineer_resources.md.*

Phase 0 · Setup & Framing (Days 1-2, ~5h)

- [ ] Internalize the 3-layer AI stack (application / model / infrastructure)
- [ ] Set up Python env, API keys (.env), cost dashboard, repo template
- [ ] Refresh: async/await, pydantic, typing, streaming, httpx
- [ ] Refresh inference concepts: tokens, context window, temperature, embeddings
- [ ] **Deliverable:** hello_llm.py — streaming + retries + token/cost logging

Phase 1 · LLM API Fluency & Prompting (Days 3-5, ~10h)

- [ ] Master one provider API: messages, streaming, tool calling, JSON mode
- [ ] Prompt engineering: system design, few-shot, CoT, output contracts
- [ ] Structured outputs with Pydantic
- [ ] First mini-eval: golden set + LLM-as-judge basics
- [ ] **Mini-project:** structured-extraction tool + small eval set

Phase 2 ★ Evaluation Foundations (Days 6-7, ~7h) — YOUR GAP

- [ ] Eval taxonomy: deterministic / semantic / rubric / human
- [ ] Metrics by task: RAG (faithfulness, relevancy, context P/R), agents, chat
- [ ] Build a golden dataset; set pass/fail thresholds (gates)
- [ ] LLM-as-judge: judge prompts, pointwise vs pairwise, pitfalls
- [ ] The correction loop: measure → diagnose → fix ONE thing → re-measure
- [ ] **Deliverable:** wrap Phase-1 tool in a DeepEval CI suite (GitHub Actions)

Phase 3 ★ RAG (Days 8-13, ~20h) — heavy, POC first

- [ ] **POC:** naive pipeline (load → chunk → embed → Chroma → retrieve → answer)
- [ ] Chunking strategies (recursive, semantic, structure-aware)
- [ ] Embeddings + vector DBs (Chroma → pgvector/Qdrant) + metadata filtering
- [ ] Hybrid search (dense + BM25) + reranking (cross-encoder / Cohere)
- [ ] Query transforms: multi-query, HyDE, decomposition, routing
- [ ] RAG evaluation with RAGAS; diagnose retrieval vs generation
- [ ] Awareness: agentic RAG, GraphRAG
- [ ] **PROJECT #1:** Chat-with-your-docs — hybrid+rerank+citations+RAGAS report, deployed

Phase 4 ★ Agents & Tool Use (Days 14-19, ~20h) — heavy, POC first

- [ ] Tool/function calling + ReAct pattern; the autonomy spectrum
- [ ] **POC:** single-tool agent (no framework) → 2-3 tool ReAct agent
- [ ] LangGraph: state, nodes/edges, routing, cycles, checkpointing, human-in-loop
- [ ] Memory (short/long term); async/queue patterns for slow LLM calls
- [ ] Agent eval: task success, tool-call correctness, trajectory, cost/latency
- [ ] Tracing with Langfuse / LangSmith to debug trajectories
- [ ] **PROJECT #2:** multi-step LangGraph agent + tracing + eval harness

Phase 5 · MCP (Model Context Protocol) (Days 20-21, ~7h)

- [ ] Learn MCP: primitives (tools/resources/prompts), host/client/server, transports
- [ ] **POC:** build an MCP server (FastMCP) exposing tools/resources
- [ ] Rewire your agent to reach tools via MCP
- [ ] Security awareness: tool poisoning, prompt injection, approval gates
- [ ] **Deliverable:** working MCP server + agent consuming it

Phase 6 ★ Fine-tuning LoRA/QLoRA (Days 22-25, ~14h) — POC, scoped

- [ ] Decision framework: when to fine-tune vs prompt vs RAG
- [ ] Concepts: PEFT, LoRA, QLoRA, ranks, target modules
- [ ] Instruction dataset curation + formatting (the hard part)
- [ ] **POC:** LoRA fine-tune a small model (transformers+peft+trl, or Unsloth)
- [ ] Evaluate vs prompt baseline; guard overfitting / forgetting
- [ ] **Deliverable:** adapter + eval report vs baseline + your recommendation

Phase 7 · Productionization & Guardrails (Days 26-27, ~7h)

- [ ] Serve with FastAPI; async/queue for long runs; streaming
- [ ] Deploy: Docker + cloud host; secrets management
- [ ] LLMOps: prompt versioning, tracing, cost tracking, caching, fallbacks
- [ ] Guardrails: I/O validation, PII, prompt-injection defense, moderation
- [ ] Monitoring: online evals + regression gates in CI/CD
- [ ] **Deliverable:** one project made production-shaped (dockerized, deployed, traced)

Phase 8 ★ Capstone (Days 28-30+, ~15-20h) — integrate everything

- [ ] Combine RAG + LangGraph agent + MCP + structured outputs + full evals + deploy + guardrails
- [ ] (Optional) include a fine-tuned component
- [ ] Share your capstone idea → get a scoped spec + architecture + eval plan
- [ ] Package: clean READMEs, architecture diagram, eval metrics, demo GIF
- [ ] Write a short blog / LinkedIn post per project

Interview Prep (threaded through the month)

- [ ] System design: design a RAG system, design an agent, tradeoffs
- [ ] Concept fluency: RAG vs FT vs prompting, hallucination mitigation, evals, MCP
- [ ] Keep DSA warm (light maintenance)
- [ ] Prepare project talking points (esp. the eval metric you moved)

Triage if you fall behind: trim fine-tuning to the POC, lighten MCP, compress deployment — but never cut Evaluation, RAG eval, Agent eval, or the Capstone.